

제 2 장 UNIX 커널 내부 구조

- 커널 내부 구조는 **사용자에게 보이지 않도록(투명하게, transparency) 설계**
→ **Data abstraction 개념으로 설계**
- 유닉스 커널이 동작 중에 이미지는 주기억장치 내에 상주
 - 프로세스 관리, 주기억장치 관리, 파일 시스템 관리, 입출력 장치 관리로 구성
- 유닉스 시스템 구조
 - Pp.37
 - 유닉스 커널의 내부 구조를 보다 자세히 정리 : pp.38

2.1 프로세스 관리

- 커널의 프로세스 관리에서 하는 기능
 - 프로세스의 생성 및 소멸
 - 프로세스 간의 통신 : IPC(InterProcess Communication)
 - CPU 스케줄링 기능
 - 제한된 자원에 대한 효율적인 관리 기능 제공
 - 프로세스 알람(alarm) 기능 제공 (CPU)
 - 프로세스 시그널(signal) 기능 제공
- 유닉스 프로세스 상태 천이도 : pp.41
 - 대기, 실행, 슬립 상태가 존재
 - 좀비(Zombie)상태 ?
---> zombie process가 가진 자원은 반드시 회수를 해야한다.
(parent process가 회수)
- 프로세스의 구성 요소
 - 텍스트(Text) 영역: 실행 가능한 프로그램 파일의 text segment가 메모리에 적재된 부분으로, 프로그램의 실행 코드가 저장되는 영역
 - 데이터(Data) 영역: 실행 파일의 data segment가 메모리에 적재된 부분으로, 전역 변수와 정적 변수가 저장되는 영역
 - 사용자 스택(User Stack): 함수(서브루틴) 호출 시 매개변수, 지역 변수, 복귀 주소 등을 저장하는 영역
 - u-area: 커널 메모리 영역에 존재하며 u-structure와 커널 스택으로 구성됨
 - u-structure: 프로세스 관리에 필요한 정보(프로세스 상태, 파일 정보 등)를 저장하는 영역
 - 커널 스택(Kernel Stack): 시스템 호출이나 인터럽트 발생 시 CPU 상태와 커널 처리 정보를 저장하는 영역
- 프로세스 수행 구조
 - **proc. 자료 구조**는 커널 내에서 언제든지 접근이 가능한 자료 구조
 - u. area는 프로세스 공간의 일부로 프로세스가 수행중일 때만 주기억장치 내에 맵되고 접근이 가능한 자료 구조

data backup
tar ~~~~~
~~
#

fork(2)

----> 대기 상태(큐)로 진입

exec(2)

----> 실행 상태로 진입

wait(2)

----> 자식 프로세스가 종료되기(zombie상태 진입)를

● **u. area 내의 주요 자료 필드**

- 프로세스 제어 블록
- 프로세스의 proc. 자료 구조를 가르키는 포인터
 - real UID, real GID, effective UID, effective GID
 - 현재 시스템 호출에 대한 인수, 복귀값 또는 에러 상태 정보
 - 시그널 핸들러 및 관련 정보
 - 프로그램 헤더에 저장된 정보들
 - open file descriptor table 정보들
 - 현재 디렉토리 및 vnode 포인터
 - CPU 사용 통계량

● **proc. table의 주요 필드**

- 프로세스 제어 블록
- 프로세스의 proc 자료 구조를 가르키는 포인터
 - real UID, real GID, effective UID, effective GID
 - 현재 시스템 호출에 대한 인수, 복귀값 또는 에러 상태 정보
 - 시그널 핸들러 및 관련 정보

기다리는 system call

----> 프로세스 동기화 기능으로 사용

(1) 프로세스 관리 정보

- 프로세스 테이블 : 프로세스의 위치, 크기, 프로세스 ID(PID), 사용자 ID(UID)
- u-area: 사용자와 그룹 ID(GID), 파일 기술자(file descriptor) 테이블, 현재 디렉토리의 i-node로 연결하는 포인터, signal처리 포인터들, CPU상태 저장(kernel stack) 등

(2) 시스템 호출 fork(), exec(), wait()

● 프로세스의 생성 : fork() 시스템 호출

- UNIX 시스템에서 프로세스는 fork 시스템 호출(system call)에 의하여만 생성
 - fork는 현재의 process(fork를 호출하는 프로세스)와 똑같은 프로세스(자식 프로세스)를 만드는 시스템 호출
 - 생성된 자식 프로세스는 open 된 파일도 공유
 - 부모 프로세스와 다른 점은 fork의 return값, 프로세스 ID, 부모 프로세스 ID(PPID)뿐이다.

● exec 시스템 호출

- UNIX 시스템에서 실행을 담당하는 시스템 호출
 - exec 호출은 프로세스가 CPU에서 실행이 될 수 있도록 한다.

^ - C : signal

● wait 시스템 호출

- wait 호출은 부모 프로세스가 자식 프로세스들의 실행 종료를 대기 상태로 기다림

(3) 프로세스 스케줄링 알고리즘

● 대화식(Interactive) ->

셸이나 문서 편집기 같은 응용 프로그램이나 GUI 환경, 프로그램은 사용자와 끊임없이 상호작용하면서 수행

- 배치(Batch) -> 작업을 한꺼번에 모아서 처리하는 것
- 실시간(Real-Time) -> 정해진 시간에 어떤 작업의 처리를 보장해주는 시스템
 - Soft real-time :
 - Hard real-time :
 - Rate-Monotonic Scheduling (RMS) 알고리즘 : 각 작업에 실행 주기를 할당하고, 주기가 짧은 작업에 더 높은 우선순위를 부여하는 간단하고 효율적인 알고리즘
- 전통적인 유닉스 스케줄링
 - 시분할, 대화식 환경을 주 대상으로 설계
 - 조건 preemptive 기능으로 설계
 - 프로세스의 우선순위에 의하여 CPU를 점유할 수 있도록 함.
 - CPU를 사용한 프로세스는 우선순위를 떨어뜨려버림 : 공정한 CPU 사용을 보장하기 위해
 - multiple-feedback queue RR with priority 스케줄링 알고리즘 사용
- 시그널과 알람
 - 유닉스 운영체제에서 제공하는 SW 인터럽트
 - 시스템에서 발생하는 비정상적인 상황을 처리하도록 설계
 - alarm : 프로세스의 동작을 정해진 시간에 제어할 수 있도록 설계
 - 커널에서 기능제공

2.2 메모리 관리

(1) 가상 메모리 관리 기능

- 제한된 메모리를 가지고 수행 프로세스 이미지 전체를 메모리에 적재하는 것이 아니라 페이지 단위로 나누어 필요한 부분만 메모리에 적재

(2) 가상 메모리의 탄생 배경

- 한정되어있는 메모리를 효과적으로 사용하기 위한 중첩(Overlay)기술
- 가상 메모리 관리 기술은 공간 낭비를 줄임
 - 수행 프로그램들이 실제적인 메모리 공간보다 클 수 있다.
 - 프로그램의 가상 메모리를 실제 메모리와 분리시킴으로써 무한대의 메모리 공간이 있는 것처럼 해줌

(3) 요구 페이징(Demand paging)

- 각 페이지는 실행중인 프로세스에 의해서 명백히 참조될 때에만 비로소 메모리에 적재함.
 - 어느 페이지를 메모리에 적재할 것인가를 결정하는데 있어서의 오버헤드를 최소화
 - 버클리 4.2BSD부터 도입

(단점) 시스템 성능이 저하

2.3 파일 시스템 관리

● 유닉스 다양한 종류의 파일 시스템을 제공

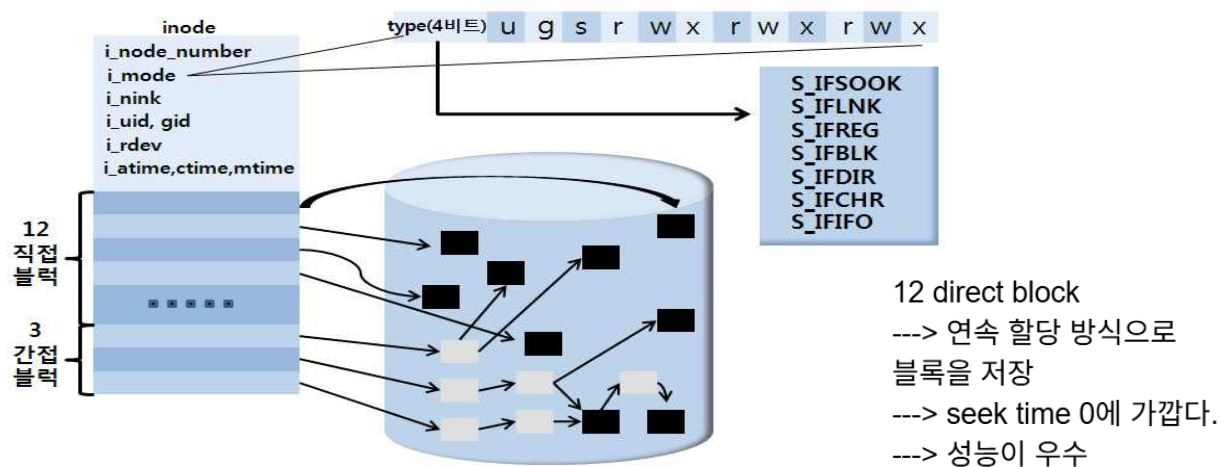
-> FFS, UFS(Unix File System), RFS, NFS, S5FS, EXT3등

(1) inode

● 파일 시스템 내에서 파일을 관리하는 자료 구조

● inode 구조

- 파일 크기,
- 사용자 식별번호, 그룹식별번호, 허가, 생성일, 마지막 변경된 날짜등
- pp.60



(2) 디렉토리

● 임의의 파일들의 집합

유닉스 파일시스템 계층구조는 디렉토리를 이용한 트리 구조

/<--- root directory
 /---> root file system으로 사용
 /etc/passwd 파일에서 정의

● pp.61

- 맨 위에는 루트 디렉토리가 있고, 아래로 내려가면서 서브디렉토리가 위치

* 홈 디렉토리 -> 사용자가 로그인 할 때 최초로 접근하는 디렉토리

(3) 사용자 계층에서 파일 접근

● 사용자 프로세스와 I-node 사이의 관계 이해

fd(file descriptor) - 파일 테이블을 가르키는 키 배열

fd[0] - 표준 입력 (stdin)

fd[1] - 표준 출력 (stdout)

fd[2] - 표준 에러 (stderr)

파일을 열때는 사용하지 않는 fd를 파일에 연결

sys_open()시 인자로 넘어간 파일 이름으로 디렉토리를 검사해 I-node를 얻어온다.

file descriptor table entry

---> 파일을 생성 또는 open

하게 되면 file descriptor table entry 할당(커널)

---> 위치값은 항상 양의 정수값

---> 0, 1, 2 : 특수 목적으로 사용

0 : standard input

\$ aaa < /tmp/1 (recirection 기능)

~~

\$

```

{
~~
scanf(); <--- standard input
~~
}

1: standard output
$ ls -lRF / > /tmp / 1
~~
$
{
~~
print(); <--- standard output
~~
}

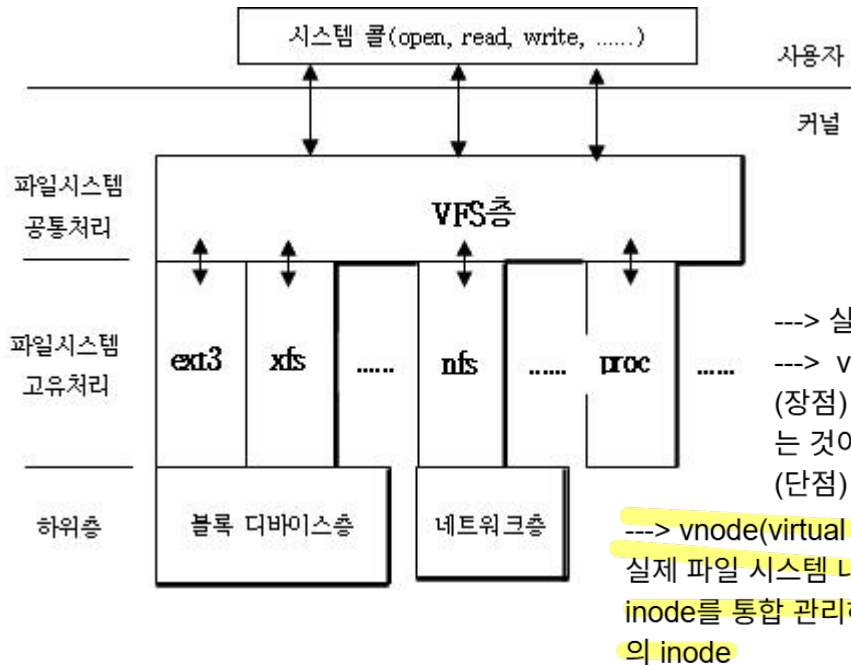
2: standard error
$ gcc -o a a.c 2> /tmp/err
~~
$

```

(4) vnode/VFS(Virtual File System)

● SunOS에서 최초로 도입

- 여러종류의 파일/ 파일 시스템을 지원하기 위한 자료구조
- 여러 타입의 파일 시스템을 동시에 지원하기 위한 구조
- 네트워크등을 통한 파일 시스템 지원을 완벽하게 보장
- 다양한 종류의 파일 시스템을 통합-관리하는 가상의 파일 시스템 -> 실제 파일 시스템 x
- 실제 파일시스템 내의 inode를 통합 관리하는 가상의 inode



(6) 수퍼블록(super-block)

● 파일시스템에 대한 모든 정보를 저장 및 관리하는 커널내의 자료구조

수퍼블록 정보	설명
inode 총수	유닉스 파일 시스템 내에 사용할 수 있는 inode의 총수
블록 총수	유닉스 파일 시스템 내에 사용할 수 있는 총 블록의 수
빈 블록수	파일 시스템 내에 현재 남아 있는 사용할 수 있는 블록의 수
빈 inode수	파일 시스템 내에 현재 남아 있는 사용할 수 있는 inode의 수
사용가능한 최초 블록 번호	유닉스 파일 시스템 내에 현재 사용 가능한 최초 블록 번호
블록 크기	유닉스 파일 시스템 내에 설정된 한 블록의 크기를 지정
마지막 마운트 시간	유닉스 파일 시스템이 마지막으로 마운트 된 시간을 저장
마지막 쓰기 시간	파일 시스템 내에 블록을 마지막으로 쓰기를 한 시간 정보를 저장
매직 번호	유닉스 파일 시스템 내의 파일의 타입을 지정하는 번호

● 유닉스에서 지원하는 파일 시스템 종류

---> ext2, ext3, ext4, xfs, btrfs, vfat, ntfs, iso9660, nfs, proc, tmpfs

- > journalling 기능을 가지고 있다.
- > file system 가용성(신뢰성)을 향상
- > 임베디드 시스템에서 주로 사용

spooler = 저장기능이 없는 장치

2.4 입출력 관리

● 블록장치(block device)

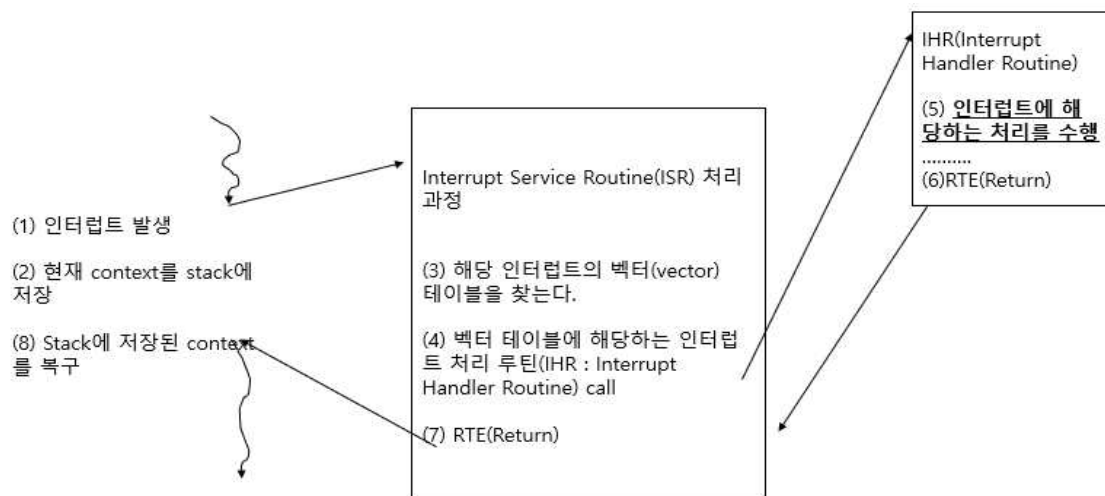
- 표준 크기인 블록 단위로 장치와 커널 사이에 데이터 이동
- HDD, DVD, USB메모리, CD-ROM, 디스크, 자기 테이프 등

● 문자장치(character device)

- 데이터 처리단위가 문자(character)크기
- 모니터, 마우스, 키보드, 단말기 등

● 인터럽트(interrupt)

- 시스템 상에서 어떤 이벤트가 발생할 경우 현재 수행중인 프로세스를 중지시키고 그 요청에 대한 작업을 수행하는 것
- pp.70 ~ 71



- 인터럽트 신호가 발생
- 현재의 프로세스는 일시정지
- 현재 프로세스의 컨텍스트(context)_를 인터럽트 스택에 저장
- 커널은 인터럽트 발생 장치를 판별
- 인터럽트 벡터 테이블을 탐색
- 인터럽트 처리기(interrupt handler)를 통해서 지정된 작업을 처리
- 인터럽트 처리가 마무리되면 인터럽트 스택에 저장된 프로세스의 컨텍스트를 복구

2.5 특수 파일(special file) 또는 디바이스 파일(device file)

● /dev 저장

● 파일 시스템들은 특수 파일로 접근이 가능하다

- open, close, read, write 시스템 호출은 프로그램 내의 특수파일과 관련하여 사용이 가능하다.
- 특수 파일은 일반 파일과는 다르게 커널 내에서 장치를 구별하는데 <major#, minor#>로 구분.
- 주 장치번호(major number), 보조 장치번호(minor number)

major number

---> IO장치를 기능적으로 구분하는데 사용하는 값

IO 장치를 파일처럼 사용 가능하도록 해준다.

minor number

---> special file

---> 같은 기능을 하는 IO 장치들간에 구분하는데 사용하는 값

2.6 버퍼 캐쉬(buffer cache)

● 유닉스 시스템은 메모리 일부분을 파일 시스템을 통해서 접근되는 블록을 캐쉬 하는데 사용

● Delayed write 또는 write behind

- 버퍼 캐쉬는 파일에 기록하게 될 변경된 데이터가 저장되어있는데 이 변경된 데이터를 즉시 블록 장치에 쓰기를 하지 않고 일정한 시간 간격으로 저장한다.
- 보조기억장치의 쓰기 연산을 줄일 수 있다.

crw-rw-rw-

b : block special

c : character special file

단점) 파일 시스템 가용성(신뢰성)이 저하될 수 있다.

example)

crw-rw-rw- 1 root sys 187,45

187, 45 (major #, minor #)